

SBI 811

Computer Programming in Bioinformatics

Lecture 1: Linux Foundations for Bioinformatics

Terminal, Shell, Paths, Files, Directories, Help and Basic Biological File Types

Level	Master of Science in Bioinformatics
Lecture emphasis	General foundations with practical bioinformatics applications
Practical environment	Linux command line
Main application	Working with FASTA, FASTQ, GFF and tabular files

Central idea: Linux is not learned as a list of commands. It is learned as an environment for organizing, inspecting, searching and processing biological data.

Learning outcomes

By the end of this lesson, you should be able to:

- Explain why Linux and the command line are widely used in bioinformatics.
- Distinguish between Linux, a terminal and a shell.
- Recognize the basic structure of a Linux command.
- Determine your current location in the Linux filesystem.
- Navigate between directories using relative and absolute paths.
- Create, copy, move, rename and remove files and directories.
- Use command-line tools to inspect text-based biological files.
- Obtain help for unfamiliar Linux commands.
- Recognize common biological data formats including FASTA, FASTQ, GFF and tabular files.
- Set up and navigate a structured bioinformatics project.

1. Why command-line computing is important in bioinformatics

Bioinformatics deals with biological information that is stored digitally. Depending on the experiment, this may include DNA sequences, RNA sequences, protein sequences, whole genomes, sequencing reads, genome annotations, sequence alignments, gene-expression measurements and many other forms of biological information.

Some biological files are small enough to inspect manually. Others can contain millions of records and occupy several gigabytes of disk space.

For example, a sequencing experiment may produce files such as:

```
sample01_R1.fastq.gz  
sample01_R2.fastq.gz  
sample02_R1.fastq.gz  
sample02_R2.fastq.gz  
sample03_R1.fastq.gz  
sample03_R2.fastq.gz
```

A project involving 100 samples may therefore contain hundreds of files. It would be inefficient to process each file manually using menus and graphical applications.

The command line allows instructions to be written as text. Those instructions can then be repeated, modified, saved, automated, applied to many files, transferred to another computer and incorporated into reproducible workflows.

For example, a command-line program may be instructed to process a file using:

```
program input.fastq
```

The same operation can later be repeated on another file:

```
program sample02.fastq
```

Eventually, these repeated operations can be automated. This ability to combine data management, software execution and automation makes Linux an important environment for bioinformatics.

2. What is Linux?

Linux is a family of operating systems. An operating system manages the basic resources of a computer and provides an environment in which other programs can run.

- running programs;
- managing memory;
- organizing files;
- controlling access to files;
- communicating with hardware;
- managing users and processes.

Other familiar operating systems include Windows and macOS. Linux is available in many distributions, including Ubuntu, Debian, Fedora, Rocky Linux and AlmaLinux.

Different distributions may differ in appearance and software-management systems, but they share many of the same command-line concepts.

Linux is commonly used in bioinformatics because it provides a powerful command-line environment, supports automation and scripting, handles large text files efficiently, allows programs to be combined, supports scientific software, and is widely used on remote servers and high-performance computing systems.

3. A short note on WSL

Students using Windows may run Linux through Windows Subsystem for Linux (WSL). WSL allows a Linux distribution such as Ubuntu to operate within Windows.

For the commands covered in this lesson, there is normally no important difference between working in Ubuntu directly and working in Ubuntu through WSL.

```
pwd
ls
cd
mkdir
```

From this point onward, the notes simply refer to the Linux environment.

4. Graphical interface versus command-line interface

Most computer users first learn to interact with computers through a graphical user interface (GUI). A GUI uses windows, menus, icons, buttons and mouse actions.

For example, to create a directory graphically, you may open a file manager, navigate to the desired location, create a new folder and enter its name. At the Linux command line, the same operation may be:

```
mkdir sequences
```

Both approaches create a directory. The major advantage of the command-line approach is that the exact instruction can be saved and repeated.

```
mkdir sequences
mkdir results
mkdir scripts
```

These commands can later become part of a script that creates an entire project automatically.

5. The terminal

A terminal is the application through which commands are entered and output is displayed. A terminal prompt may look like:

```
student@computer:~$
```

The exact appearance varies between computers. The dollar sign normally indicates that the terminal is ready to receive a command.

6. What is a shell?

The terminal and shell are related but are not exactly the same. The terminal provides the interface in which commands are typed. The shell interprets those commands.

One of the most widely used Linux shells is Bash. Bash stands for Bourne Again Shell.

```
User
|
v
Terminal
|
v
Shell
|
v
Linux
```

Suppose you type:

```
ls
```

The shell interprets `ls` as a command and asks Linux to execute the appropriate program. The result is returned to the terminal.

7. Basic structure of a Linux command

Linux commands commonly follow the pattern:

```
command options arguments
```

Consider:

```
ls -l data
```

This can be interpreted as: ls = command, -l = option, data = argument.

The command tells Linux what operation to perform. The option modifies how the operation should be performed. The argument specifies what the command should operate on.

Another example is:

```
head -n 5 genes.fasta
```

Here, head is the command, -n 5 is the option, and genes.fasta is the argument. The instruction means: display the first five lines of genes.fasta.

8. The Linux filesystem

Files in Linux are organized hierarchically. The highest level is the root directory, represented by a single forward slash (/). Everything else occurs somewhere beneath it.

```
/
|-- bin
|-- etc
|-- home
|   |-- student
|       |-- Documents
|       |-- Downloads
|       |-- bioinformatics
|-- tmp
|-- usr
```

9. Home directory

Each user normally has a personal home directory. For a user named student, this may be:

```
/home/student
```

The symbol ~ is commonly used to represent the current user's home directory. Therefore ~/bioinformatics may represent /home/student/bioinformatics.

10. Current working directory

At any moment, the shell considers the user to be located in a particular directory. This is called the current working directory.

Suppose you are located in:

```
/home/student/project/data
```

Commands using relative paths are interpreted from this location. It is therefore important to know where you are before manipulating files.

11. Files and directories

A file stores information. Examples of files in bioinformatics include:

```
genome.fasta
reads.fastq
annotation.gff
alignment.bam
variants.vcf
counts.tsv
```

A directory is used to organize files and other directories.

```
project/
|-- data/
|-- reference/
|-- scripts/
`-- results/
```

12. Paths

A path specifies the location of a file or directory. For example:

```
/home/student/project/reference/genome.fasta
```

Linux separates directories using the forward slash (/).

12.1 Absolute paths

An absolute path begins from the root directory /. It specifies the complete location of a file.

```
/home/student/project/reference/genome.fasta
```

12.2 Relative paths

A relative path is interpreted from the current directory. Suppose the current directory is /home/student/project. The same file can be referred to as:

```
reference/genome.fasta
```

Relative paths are useful because they make project structures more portable.

13. Special path symbols

Symbol	Meaning
/	Root directory
~	Home directory
.	Current directory
..	Parent directory

For example, ../results means: move one level upward and then access the results directory.

14. Case sensitivity

Linux is case-sensitive. Therefore the following represent different filenames:

```
genes.fasta
Genes.fasta
GENES.fasta
```

Correct spelling and capitalization are therefore important when working at the command line.

15. File naming in bioinformatics

Bioinformatics projects commonly contain many related files. Good filenames should be informative, consistent, concise and easy to process computationally.

```
sample01_R1.fastq.gz
sample01_R2.fastq.gz
sample02_R1.fastq.gz
sample02_R2.fastq.gz
reference_genome.fasta
annotation.gff
gene_counts.tsv
```

Spaces in filenames are best avoided. Instead of 'sample one reads.fastq', prefer 'sample_one_reads.fastq' or 'sample-one-reads.fastq'.

16. Biological file formats

Bioinformatics programs communicate through structured file formats. Different formats represent different biological information.

Format	Common extension	Main content
FASTA	.fa, .fasta, .fna, .faa	DNA, RNA or protein sequences
FASTQ	.fastq, .fq	Sequencing reads and quality scores
GFF/GTF	.gff, .gff3, .gtf	Genome annotations
SAM/BAM	.sam, .bam	Sequence alignments
VCF	.vcf	Genetic variants
TSV	.tsv	Tab-separated tables
CSV	.csv	Comma-separated tables

17. FASTA format

FASTA stores biological sequences. Each FASTA record normally contains a header followed by a sequence.

```
>gene1
ATGCGTACGTTAGCTAGCTAGC
>gene2
ATGCCGACGTTAGATAGCTAGC
```

The header begins with >. FASTA can contain nucleotide or protein sequences.

```
>protein_1
MKKIGYSAPRQT
```

18. FASTQ format

FASTQ is widely used for raw sequencing data. A FASTQ record normally contains four lines:

```
@read1
ATGCGTACGT
+
IIIIIIIIII
```

These contain the read identifier, nucleotide sequence, a separator, and quality scores. FASTQ therefore differs from FASTA because it contains information about sequencing quality.

19. GFF format

A genome sequence alone does not specify where genes are located. Genome annotation formats such as GFF describe biological features and their coordinates.

chr1	RefSeq	gene	100	500	.	+	.	ID=gene1
chr1	RefSeq	gene	800	1200	.	-	.	ID=gene2

A GFF file contains several tab-separated columns. Detailed manipulation of these columns will be introduced later, but basic Linux tools can already be used to inspect and search them.

20. Tabular data

Many bioinformatics analyses produce tables. For example:

gene_id	sample1	sample2
geneA	156	187
geneB	34	29
geneC	816	901

This may be stored as `gene_counts.tsv`. TSV means tab-separated values. Tabular data are particularly suitable for command-line processing because individual columns and rows can be extracted easily.

21. Organizing a bioinformatics project

A bioinformatics analysis should be organized from the beginning. A useful project structure is:

```
rna_seq_project/
|-- data/
|   |-- raw/
|   `-- processed/
|-- reference/
|-- scripts/
|-- results/
|-- logs/
`-- README.md
```

data/raw: contains original experimental data such as `sample01_R1.fastq.gz` and `sample01_R2.fastq.gz`. Original data should normally remain unchanged.

data/processed: contains data derived from raw data.

reference: contains reference sequences and annotations such as genome.fasta and annotation.gff.

scripts: contains analysis commands and programs.

results: contains important outputs.

logs: contains execution messages, warnings and errors.

README.md: explains the purpose and organization of the project.

APPLICATION: LINUX FOR BIOLOGICAL DATA

The following practical uses small FASTA, FASTQ, GFF and count files. The aim is to see Linux as a tool for answering biological-data questions rather than only as a file-management environment.

22. Create the working project

```
cd ~
pwd
mkdir -p SBI811/week1_project/{data/raw,data/processed,reference,scripts,results,logs}
cd SBI811/week1_project
pwd
ls
ls -lh
```

The project now has separate locations for raw data, processed data, reference files, scripts, results and logs.

23. Create a FASTA dataset

Create the file reference/genes.fasta using a text editor such as nano and enter:

```
>geneA hypothetical_protein
ATGCGTACGTTAGCTAGCTAGCTAGCTA
>geneB transporter_protein
ATGGCGACGTTAGATAGCTGGCTAACGA
>geneC hypothetical_protein
ATGAAAACGCTAGCTAGCTAGCTAGTAA
>geneD regulatory_protein
ATGCCCGTTAGCTGGCTAACTAGCTGA
>geneE transporter_protein
ATGCGTACGTAGGCTAGCTAGGCTGTAA
```

24. Basic inspection of FASTA

```
cat reference/genes.fasta
head reference/genes.fasta
tail -n 4 reference/genes.fasta
wc -l reference/genes.fasta
```

These commands allow the file to be displayed, partially inspected and counted without opening a graphical application.

25. Use grep to search biological information

The grep command searches text for a pattern.

```
grep "transporter" reference/genes.fasta
```

This returns the FASTA headers annotated as transporter proteins.

```
grep "hypothetical" reference/genes.fasta
```

Count the number of matching lines:

```
grep -c "hypothetical" reference/genes.fasta
```

26. Finding all FASTA headers

Every FASTA header begins with >. Therefore:

```
grep ">" reference/genes.fasta
```

A safer expression explicitly requires > to occur at the beginning of the line:

```
grep -c "^>" reference/genes.fasta
```

The result is the number of FASTA records. This is a useful example of using Linux to answer a biological-file question directly.

27. Create a FASTQ dataset

Create data/raw/sample01_R1.fastq containing:

```
@read1
ATGCGTACGT
+
IIIIIIIIII
@read2
ATGCGTTCGT
+
IIIIIIHIII
@read3
TTGCGTACGA
+
IIIIIGIII
@read4
ATCCGTACGT
+
IIIIIIIIII
```

28. Inspect FASTQ

```
head -n 4 data/raw/sample01_R1.fastq
head -n 8 data/raw/sample01_R1.fastq
wc -l data/raw/sample01_R1.fastq
```

The file contains 16 lines. Because one conventional FASTQ record occupies four lines, it contains four reads.

```
expr $(wc -l < data/raw/sample01_R1.fastq) / 4
```

At this stage, the full syntax does not need to be memorized. The important point is that command-line tools can convert file structure into useful information.

29. Create a GFF annotation file

Create reference/annotation.gff containing:

```
##gff-version 3
chr1    RefSeq    gene      100      500      .      +      .      ID=geneA
chr1    RefSeq    CDS       100      500      .      +      0      ID=cdsA
chr1    RefSeq    gene      700      1200     .      -      .      ID=geneB
chr1    RefSeq    CDS       700      1200     .      -      0      ID=cdsB
chr1    RefSeq    gene      1500     1850     .      +      .      ID=geneC
chr1    RefSeq    gene      2100     2600     .      -      .      ID=geneD
chr1    RefSeq    CDS       2100     2600     .      -      0      ID=cdsD
```

30. Search the GFF with grep

```
grep "gene" reference/annotation.gff
grep -c "CDS" reference/annotation.gff
grep $'\t-\t' reference/annotation.gff
```

These commands demonstrate how text searching can reveal feature types and strand information.

31. Introducing awk through a biological question

A GFF file contains tab-separated fields. The third field contains the feature type. We can print the third field using:

```
awk '{print $3}' reference/annotation.gff
```

The header line ##gff-version 3 also contributes a value, illustrating why biological file structure matters when interpreting output.

Remove comment/header lines first:

```
grep -v "^#" reference/annotation.gff
```

Now combine the two commands:

```
grep -v "^#" reference/annotation.gff | awk '{print $3}'
```

32. Count the number of genes in GFF

```
awk '$3=="gene"' reference/annotation.gff
awk '$3=="gene"' reference/annotation.gff | wc -l
```

The first command displays only records whose third column is exactly gene. The second counts them.

33. Extract gene coordinates

```
awk '$3=="gene" {print $1, $4, $5}' reference/annotation.gff
awk '$3=="gene" {print $1, $4, $5, $7}' reference/annotation.gff
```

These commands extract chromosome, start, end and strand information for gene features.

34. Calculate gene lengths with awk

For inclusive genomic coordinates, gene length is calculated as end - start + 1.

```
awk '$3=="gene" {print $9, $5-$4+1}' reference/annotation.gff
```

This demonstrates that Linux can perform simple calculations directly from structured biological data.

35. Create a gene count table

Create results/gene_counts.tsv containing:

gene_id	sample01	sample02
geneA	120	145
geneB	45	51
geneC	803	764
geneD	12	19
geneE	203	212

```
cat results/gene_counts.tsv
```

36. Extract columns with cut

```
cut -f1 results/gene_counts.tsv  
cut -f1,2 results/gene_counts.tsv
```

The -f option selects fields. For a TSV file, fields are separated by tabs.

37. Find highly expressed genes with awk

Suppose we want genes with more than 100 counts in sample01.

```
awk '$2 > 100' results/gene_counts.tsv  
awk 'NR>1 && $2>100' results/gene_counts.tsv
```

The second form excludes the header row and returns only data rows meeting the threshold.

38. Sort genes by count

```
tail -n +2 results/gene_counts.tsv  
tail -n +2 results/gene_counts.tsv | sort -k2,2n  
tail -n +2 results/gene_counts.tsv | sort -k2,2nr
```

The final command sorts numerically from highest to lowest according to the second column.

39. Combining Linux commands

One of the major strengths of Linux is that commands can be connected. The symbol | is called a pipe. It sends the output of one command to another command.

```
grep -v "^#" reference/annotation.gff | awk '{print $3}'
```

This can be read as: remove comment lines, then extract column 3.

Continue the pipeline:

```
grep -v "^#" reference/annotation.gff |  
awk '{print $3}' |  
sort |  
uniq -c
```

The pipeline answers: what feature types occur in the annotation, and how many of each are present?

40. A simple Linux bioinformatics pipeline

```
grep -v "^#" reference/annotation.gff |  
awk '{print $3}' |  
sort |  
uniq -c |  
sort -nr
```

Step 1: remove comment/header lines.

Step 2: extract the feature-type column.

Step 3: place identical feature types together.

Step 4: count them.

Step 5: sort counts numerically from highest to lowest.

A complicated-looking Linux command is therefore often a sequence of small operations.

41. Redirecting output to a file

Linux commands normally print output to the terminal. The symbol > redirects the output into a file.

```
grep "^>" reference/genes.fasta > results/fasta_headers.txt  
cat results/fasta_headers.txt
```

Similarly:

```
awk '$3=="gene"' reference/annotation.gff > results/genes.gff
```

This creates a new file containing only gene features.

42. Practical biological questions

Using the files created above, answer the following using Linux commands rather than manually counting the records.

FASTA

- How many sequence records are present?
- How many are annotated as hypothetical proteins?
- How many are annotated as transporters?
- Display only the FASTA headers.

FASTQ

- How many lines are present?
- How many reads are present?
- Display only the first read.
- Display only the first two reads.

GFF

- How many gene features are present?
- How many CDS features are present?
- Which genes occur on the negative strand?
- Extract the start and end coordinates of all genes.
- Calculate the length of each gene.

Count table

- Display only gene IDs.
- Display gene IDs and sample01 counts.
- Identify genes with sample01 counts greater than 100.
- Sort genes from highest to lowest sample01 count.
- Identify the gene with the highest count.

43. What should be learned at this stage?

You should understand the basic operations:

```
pwd
ls
cd
mkdir
cp
mv
rm
head
tail
cat
less
wc
```

You should also begin recognizing the purpose of:

```
grep
cut
sort
uniq
awk
|
>
```

You are not expected to have mastered all grep or awk syntax yet. The important observation is that Linux commands allow biological files to be searched, filtered, summarized and transformed without manually opening them.

44. End-of-lesson application

You receive a directory containing:

```
bacteria.fasta  
bacteria.gff  
sample_R1.fastq  
sample_R2.fastq  
counts.tsv
```

Without using a spreadsheet program or graphical text editor:

- Create an appropriate bioinformatics project structure.
- Move the FASTQ files to the appropriate location.
- Move the FASTA and GFF files to the appropriate location.
- Place the count table in an appropriate results directory.
- Determine the number of sequences in the FASTA file.
- Determine the number of reads in sample_R1.fastq.
- Determine the number of gene features in the GFF file.
- Extract the coordinates of those genes.
- Determine which feature types occur in the GFF file and how many of each exist.
- Identify the genes with counts greater than a specified threshold.
- Save the list of those genes into a new results file.
- Display the commands you have recently used.

45. Summary

Linux provides a text-based environment in which files, programs and analyses can be controlled through commands. In bioinformatics, the command line is valuable because biological data are often large, structured and repetitive.

The central relationships from this lesson are:

```
Filesystem -> directories -> files -> paths -> commands  
  
FASTQ  -> sequencing reads  
FASTA  -> biological sequences  
GFF    -> genome annotation  
SAM/BAM-> sequence alignments  
TSV/CSV-> tabular results
```

The main practical principle is that Linux commands can be combined to answer biological-data questions. Simple tools such as grep, cut, sort, uniq and awk become especially powerful when used together through pipes and redirection.